

Plan Praktikum Peter 2026

1. Vorüberlegungen

1.1 *Benutzerinteresse*

- ***Welche Fragen stellen Studieninteressierte und Erstsemestler am häufigsten***
 - “Welcher Studiengang passt zu mir”
 - “Wie läuft ein Studium an der HTWK ab?”
 - “Welchen Unterschied gibt es zwischen einer Uni und einer Hochschule?”
 - “Was kann ich mit dem Studiengang (Print) später beruflich machen?”
 - “Was bedeutet der Begriff”
 - Fragen zum Unterrichtsinhalt (z.B. im Bereich Print und Verpackungen)
 - Spezifische HTWK-Fragen (zum Gelände, Metainformationen etc)
- ***Welchen Mehrwert soll der Chatbot dem Endnutzer bringen***

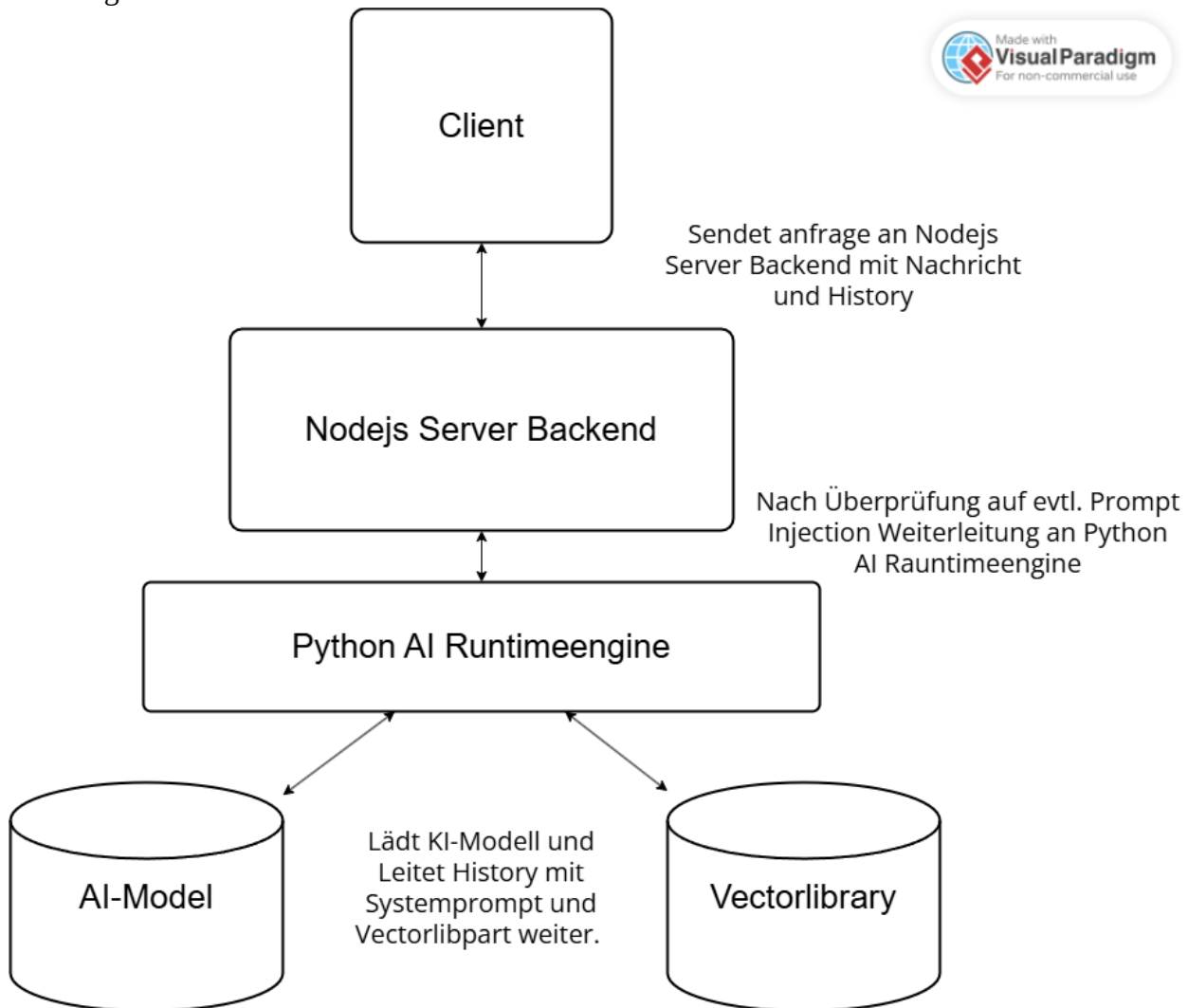
Der Chatbot soll als eine außerschulische Ansprechperson dienen, der Fragen und Informationen sicher beantwortet und sich dabei auf sichere Fakten von offiziellen externen Dokumenten beruft. Das kann sowohl Studierenden als auch Profs unterstützen bzw. entlasten.
- ***Wie viele Bereiche soll der Chatbot abdecken***
 - Print
 - Verpackung
 - Studieneinstieg
 - Studienalltag
 - HTWL-domainspezifische Informationen (z.B. Gebäude oder Professoreninfos)
 - (KI-training)

1.2 Bibliotheken und Runtime

- Welche Sprachen und Runtimes nutze ich?

Runtime	Language	Nutzen
Python virtual Environment	Python	Backend für AI-Model
NodeJS (Chromium)	Javascript	Server Backend für Client
Javascript Frontend	HTWML, Javascript, CSS	Website und Interface für Benutzer

Das Folgende soll den Prozess einer Nachricht auf dem Backend visualisieren:



- **Welche Bibliotheken nutze ich und brauche ich?**

NodeJS:

Bibliothek	Versionsnummer	Nutzen	Quellenverweis
npm:fastify	9.1.3	Dient als Runtimeengine für den Server, verwaltet PORTS und leitet Anfragen weiter. Es dient als eine moderne Alternative zu veralteten Nodebibliotheken.	https://www.npmjs.com/package/fastify
npm:@fastify/static	5.8.5	Dient als Sublibrary für die Website. Sorgt dafür, dass man eine Weiterleitung unter einer URI direkt unter einen Ordner (./public) festlegen kann.	https://www.npmjs.com/package/@fastify/static

Python (die wichtigsten):

Bibliothek	Versionsnummer	Nutzen	Quellenverweis
pip:flask	3.1.3	Dient als Serverruntime um Anfragen entgegenzunehmen.	https://pypi.org/project/Flask/
pip:waitress	3.0.2	Dient als Unterstützung für Flask. Lässt Flask Serveranfragen parallel bearbeiten (threading) um die Geschwindigkeit in hohem Tempo zu bearbeiten.	https://pypi.org/project/waitress/
pip:torch	2.8.0	Dient als Runtime für das KI-Modell und als Herzstück des gesamten Projektes. Lädt das KI-Modell, finetuned es und führt es aus. Pytorch ist weltweit mit die wichtigste Opensource-Bibliothek für KI und maschinelles Lernen.	https://pypi.org/project/torch/ Und (official): https://pytorch.org
pip:transformers	4.57.1	Transformers ist das klassische Format für KI-Modelle und unterstützt und ermöglicht erst, dass Modell so einfach zu laden.	https://pypi.org/project/transformers/ Und (für Installation): https://huggingface.co/docs/transformers/de/installation
pip:datasets	4.3.0	Datasets ist der Standardweg zum Laden, Verarbeiten und Nutzen großer Datenmengen wie z.B. Trainingsdaten für das kommerzielle KI-training.	https://pypi.org/project/datasets/ Und: https://huggingface.co/docs/datasets/installation

1.3 IT-Sicherheit

- **Wie sichere ich den Server vor Angreifern?**
 - Keinen externen Code auf den Server ausführen oder gar hochladen lassen!
 - Sicherheitsupdates haben Vorrang vor Betrieb
 - Keinen externen Zugriff auf Bootrechte
 - KI keine Möglichkeit geben, Befehle oä. auszuführen
- **Wie schütze ich den Chatbot und Nutzer vor Prompt-Injection?**
 - Keine Systemprompts auf Clientside → Systemprompt nur auf Backend HARDCODEN
 - KI keine Möglichkeit geben, Befehle oä. auszuführen
 - Evtl. rechtliche Sicherheitswarnung für Nutzer
- **Wie viele Freiheiten gebe ich den Nutzer?**
 - Reine Textnachrichten senden
- **Gibt es eine langzeitarchivierte Chathistory und wenn ja wo ist sie gespeichert?**

Der Chatverlauf wird im Runtimestorage des Browsers unter einer einfachen Variable im Klartext gespeichert werden. Das heißt bei einem Neuladen der Seite wird der Chat weg sein. Der Chatbot soll keine Multichatfunktion oder eine langzeitarchivierte Chathistory aus reinen Sicherheits- und Speicher- bzw. account- und datenrechtlichspezifischem Bedenken bieten.

1.4 Hosting bzw. Serverbetrieb

- **Wo soll der Chatbot verfügbar sein (Internet oder HTWK-eigenes Intranet)?**

Die Frage ist für die Planung erst einmal nicht relevant, da es egal wovon man zugreifen kann eine hohe Sicherheit bieten soll. Bei beiden Varianten benötigt man ein Server und die Weiterleitung von der Domain der HTWK zum Server oder vom Gateway des Intranets zum Server lässt sich je nach belieben einrichten.
- **Auf welchen Server läuft der Chatbot?**

Voraussichtlich auf einen lokalen Server in der HTWK-Leipzig
- **Wie viele Ressourcen stehen dem Betrieb zur Verfügung**

--Unklar--

1.5 Rechtliches und Lizenzen

- Unter welchen Rechten läuft die Nutzung des gewählten Modells (welche Lizenzen)?

Apache 2.0 – kommerziell nutzbar und frei modifizierbar nur unter Angabe der Domain im Sourcecode und in der Veröffentlichung des daraus trainierten Modells (lesen sie hier: <https://www.apache.org/licenses/LICENSE-2.0.txt>)
- Unter welchen Rechten läuft die Nutzung von Trainingsdaten (welche Lizenzen)?

--in Bearbeitung--

1.6 Basismodellauswahl

- Welches Modell eignet sich gut für alle Anforderungen als Basismodell?
Folgendes Modell eignet sich gut: Huggingface:Qwen/Qwen3-4B-Instruct-2507
Hier finden Sie das Modell: <https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507>
- Wie leistungsstark ist das betrachtete Modell?
Das Modell hat 4.8 Milliarden Parameter, 36 Transformerlayers, 32 Q- und 8 KV-Attention-Heads und schneidet an der Spitze sehr gut ab.
- Wie gut liegt das betrachtete Modell im Ranking gegenüber anderen?
 - Beim Benchmarking [DistilLabs](#) belegte es unter 12 weiteren KI-Modellen den 1. Platz.
 - Folgendend gut ist das Modell von 100% (perfekt):

Disziplin	Score in %	Einordnung
Mathematik (AIME25)	83.3	Absolutes Top-Niveau für SLMs
Wissenschaften (GPQA)	65.8	Weit über Klassendurchschnitt
Coding (Livecodebench)	55.2	Führen im Opensource Bereich

- Wie viele Parameter hat es und welche Hardwareanforderungen sind benötigt?
8-12 GB VRAM sind die Mindestvorderung. Das sollte mit einer NVIDIA GeForce RTX 5090 mit 32 GB VRAM kein Problem sein und gut für den Serverbetrieb laufen.

1.7 Aufwand und Risiken

- Wie viel Aufwand bringt die Planung?
Nicht all zu viel ich denke relativ schnell.
- Wie viel Aufwand bringt das Finetuning?
Max. zwei Wochen. Natürlich ist stark davon abhängig wie viel man täglich an den Chatbot arbeitet.
- Wie viel Aufwand bringt das Hosting und die Infrastruktur?
Kommt auf die Gegebenheiten an aber ich denke ein bis drei Wochen.
- Wie viel Aufwand bringt die Instandhaltung
Einiges also wöchentlich sollte jemand mal drüber schauen um Softwarefehler zu finden und Updates zu installieren.
- Welche potentiellen Risiken gehen mit dem Plan einher?
Nicht viele bis keine.

2. Datagrapping

2.1 Ziel:

Das Erlernen von genauen Informationen zur der HTWK Leipzig.

A Reines Finetuning:

2A.1 Übersicht:

Ansammeln von DS-Daten (Domain Specific bzw. HTWK Daten) und das Umwandeln derer in Chat-Templates durch ein lokales Modell.

2A.2 Ablauf:

1. Sammeln von HTWK-spezifischen Daten (Vorlesungen etc.)
2. Semantische Aufteilung dieser Daten
3. Generierung von Trainingsdaten im Chat-Template Format durch ein lokales Modell
4. Überfliegen der Trainingsdaten durch eine Person bzw. ein lokales Modell (→ evtl. Verbesserung)
5. Splitten der Trainingsdaten in Trainings- und Validierungsdaten
6. Finetuning

2A.3 Produkt:

Modell **Version 0.1.0** mir spezifischen Daten in den Gewichten des Modells

B Vektordatenbank:

2B.1 Übersicht:

Ansammeln von DS-Daten (Domain Specific bzw. HTWK Daten) und das erstellen einer Vektordatenbank mithilfe eines lokalen Modells.

2B.2 Ablauf:

1. Sammeln von HTWK-spezifischen Daten (Vorlesungen etc.)
2. Semantische Aufteilung dieser Daten
3. Generierung einer Vektordatenbank durch ein lokales Modell (z.B. bge-m3)
4. Splitten der Trainingsdaten in Trainings- und Validierungsdaten

2B.3 Produkt:

Vektordatenbank mit spezifischen Daten

3. Schreiben von Chat-Templates

3.1 Ziel:

Das Erlernen der richtigen Form der Antwort und dem richtigen Umgang bzw. Ausdruck mit dem Nutzer.

3.2 Übersicht:

Das manuelle Schreiben von Trainingsdaten im Chat-Template-Format.

3.3 Ablauf:

1. Definieren der Eigenschaften des Chatbots
2. Auf *Schritt 1*. Basierende Planung der Reihenfolge (nach dem Prinzip wichtigstes zuerst und zuletzt)
3. Schreiben der Trainingsdaten
4. Splitten der Trainingsdaten in Trainings- und Validierungsdaten
5. Finetuning des KI-Modells (**Version [2.1] 0.1.0 [2.2] 0.0.0**) in eventueller Beachtung der Vektordatenbank abhängig von 2.x

3.4 Datenformat:

Die Trainingsdaten liegen im Chattemplateformat in einer „all.jsonl“-Datei und behalten folgende Inhaltsstruktur inne:

```
{ "messages": [
  { "role": "system", "content": "[Systemprompt]" },
  { "role": "user", "content": "[Beispielfrage]" },
  { "role": "assistant", "content": "[Idealantwort]" },
  { "role": "system", "content": "[Systemprompt]" },
  { "role": "user", "content": "[Beispielfrage]" },
  { "role": "assistant", "content": "[Idealantwort]" },
  ...
]}
```

Ziel beim Training ist der KI Beispiele zu geben, dabei wird der KI ein Dialog gegeben, die KI muss dann versuchen das letzte „assistant“-Label zu beantworten, dann wird ein Loss (Abweichung) berechnet, der genutzt wird um die KI rekursiv anzupassen. Diesen Vorgang nennt man KI Training bzw. Finetuning.

3.5 Produkt:

Modell **Version [2.1] 0.2.0 [2.2] 0.1.0**

4. KI – Aber wie?

4.1 Was ist KI?:

Eine künstliche Intelligenz kurz KI oder im englischen AI ist ein neuronales Netzwerk in Form eines Algorithmus. Sie enthält sogenannte Gewichte als Gradienten, Skalare, Matrizen bzw Tensoren, die bestimmen aus welchen Eingaben welche Ausgaben generiert werden. Wie gerade erwähnt verarbeitet eine KI Daten und da die Gewichte verändert werden können lässt sich eine KI trainieren, also das Anpassen der Gewichte zum Erfüllen bestimmter Tätigkeiten meist basierend auf gewissen Trainingsdaten.

4.2 Tokenizing:

Da ich nicht auf jeden Typ künstlicher Intelligenz eingehen kann, werde ich mich nur auf rekursive Transformer-Chatbot beschränken. Eine KI ist ja bekanntlich dazu da eine Antwort zu generieren. Damit eine KI einen Inputtext überhaupt verstehen kann, muss die KI den Text zerlegen in so genannte Tokens. Tokens sind gut vergleichbar mit einzelnen Wörtern und Satzzeichen bzw. Sonderzeichen. Zunächst sind die Tokens einfache Zahlen (pro Tokentyp ein Index), aber damit die KI jedes Wort als Bedeutung sieht müssen wir jedem Token Eigenschaften geben. Hier kommen Parameter ins Spiel. Die Anzahl an Parameter ist nämlich äquivalent zu der Anzahl der möglichen Eigenschaften pro Wort bzw. die Dimensionsgröße eines Tokenvektors. Hier ein Beispiel, wie das Aussehen könnte:

Erdbeere
Substantivartig: 0.98
Großgeschrieben: 0.9998
Lebensmittel: 0.999
Werkzeug: 0.001
Sommer: 0.639
Pflanze: 0.849
Weltraum: 0.00039
Marmelade: 0.83

So könnte eine Eingabe in Tokens aussehen:

Eingabe im Klartext	Eingabe in Tokens ([TOKEN] ist ein Token)
Hallo, kannst du mir bitte ein paar Informationen zu HTWK Leipzig auflisten?	[HALLO][,][\n][kannst][du][mir][bitte][ein][paar] [Informationen][zur][H][T][W][K][L][e][i][p][z][i] [g][auflisten][?]

Das ist natürlich nur beispielhaft aber es zeigt, wie die KI auf einmal eine Art „Gefühl“ bekommt für Wörter und diese überhaupt miteinander Vergleichen kann.

4.3 Rekursive Vervollständigung:

Wenn der Nutzer eine Nachricht zu der KI sendet passiert im Hintergrund einiges. Der neue Prompt wird an den Systemprompt und der gesamten History (der Bisherige Chat) angehängt. Das sieht dann in etwa so aus:

```
<system>Du bist ein freundlicher, hilfsbereiter und genauer Chatbot.</system>
<user>Hallo! Wer bist du?</user>
<assistant>Hallo, ich bin Scientia ein hilfsbereiter Chatbot.</assistant>
<user>Was ist  $1 + 2 * 10 ^ {2000000}$ ?</user>
```

Systemprompt
History
Message

Dies zusammengesetzte Eingabe wird zunächst in eine Tokensequenz umgewandelt. Das KI-Modell versucht jetzt diesen Input zu ergänzen und zwar um eine Antwort. Z.B:

```
<assistant>Leider kann ich dir diese Zahl nicht ausschreiben,
                aber du kannst sie dir wie folgt vorstellen:
Eine Zwei mit 1999999 Nullen und einer Eins zum Schluss.</assistant>
```

Antwort

Dafür wird dem Modell die Eingabe als Tokensequenz gegeben und es hängt ein passendes Token an. Das Token wird aus den Tokenpool gezogen. Jedenfalls wird das generierte Token an die Eingabe UND Ausgabe angehängt. Das ganze wiederholt sich: Der Chatbot vervollständigt um einen Token und hängt ihn an die Ein- und Ausgabe an. Das macht er so lange, bis er ein eos-Token (End of Sequence) zurückgibt. Wenn er dieses Token mit Zufall aus dem Pool zieht, ist die Ausgabe zu Ende, die Ausgabe wird zurückgegeben und der Benutzer bekommt seine Antwort.

Hier ist ein Beispiel, das einen solchen Vorgang beispielhaft zeigen soll:

Stage 0:

```
<user>Hi</user>
<assistant>
```

Stage 1:

```
<user>Hi</user>
<assistant>Hi
```

Stage 2:

```
<user>Hi</user>
<assistant>Hi,
```

Stage 3:

```
<user>Hi</user>
<assistant>Hi, ich
```

Stage 4:

```
<user>Hi</user>
<assistant>Hi, ich bin
```

Stage 5:

```
<user>Hi</user>
<assistant>Hi, ich bin Scientia
```

Stage 6:

```
<user>Hi</user>
<assistant>Hi, ich bin Scientia</assistant>
```

Stage 7:

```
<user>Hi</user>
<assistant>Hi, ich bin Scientia</assistant> <eos>
```

4.4 Das Training:

Das Modell besteht aus vielen Gewichten, die die aus einer Tokensequenz ein Token generieren. Wie oben schon erwähnt sind dies Gewichte nicht statisch also veränderbar. Wenn wir die Gewichte anpassen, verändern wir das Token, dass am wahrscheinlichsten gezogen wird und damit verändern wir das Verhalten des Modells. Jetzt können wir bei Milliarden von Zahlen innerhalb der Gewichte aber nicht einfach per Hand ändern. Deshalb gibt es ein Verfahren namens AI-Training. AI-Training nutzt ein Trainingsdatensatz und passt ein KI-Modell (das Basismodell) an den Trainingsdatensatz an. Meist gilt dabei, dass die KI den Trainingsdaten so genau wie möglich nachahmen soll.

Beim Training gibt es zwei Möglichkeiten:

A. Als Basismodell ein zufällig generiertes Modell nutzen

B. Als Basismodell ein schon existierendes Modell nutzen → Das nennt man Finetuning

Beispiel: Angenommen wir haben folgenden Trainingsdatensatz:

```
{"messages":[
{"role":"system","content":"Du bist eine KI"},
{"role":"user","content":"Hallo"},
{"role":"assistant","content":"Hallo! Ich bin Scientia."}
...
]}
```

Dann nehmen wir erste Trainingsdata und geben den KI-Modell die User und System Message. In unserem Beispiel würde das KI-Modell das bekommen:

```
<system>Du bist eine KI</system>
<user>Hallo</user>
<assistant>
```

Nun versucht die KI zu vervollständigen also eine Antwort zu generieren z.B.:

```
<assistant>Tschüss! Ich bin Baum?</assistant>
```

Da die Antwort des KI Modells offensichtlich stark von der Idealantwort aus der Trainingsdata abweicht, ist der *Loss* (die *Abweichung* zwischen **generierter Antwort** und **Trainingsdatenantwort**) sehr hoch.

Anhand des Loss-Wertes werden die Gewichte im Modell korrigiert und zwar so stark wie auch die *learning_rate* (Feinheit des Lernens) ist. Das ganze Prozedere wird mit allen Trainingsdaten gemacht und mehrmals (um den Faktor *num_train_epos*) wiederholt. Irgendwann kommt dann auch das richtige Ergebnis raus:

```
<assistant>Hallo! Ich bin Scientia.</assistant>
```

5. Präsentation

4.1 Ziel:

Das Präsentieren meiner Ziele, Arbeitsweise und Ergebnisse, sowie das Einführen in das KI-Finetuning

4.2 Mindestbenötigter Inhalt:

- Mein Ziel
- Meine Arbeitsweise
- Meine Probleme und deren Lösungen
- Das Ergebnis
- Wie funktioniert ein Transformermodel bzw. Chatbot grundsätzlich
- Wie funktioniert das KI-Training
- Wie kann man es weiterentwickeln – Zukunftsausblicke

6. Quellen:

- Eigene Erfahrung und eigenes Wissen
- pypi.com
- huggingface.co
- npmjs.com
- apache.org